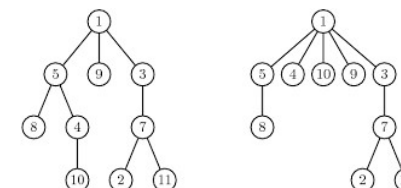


Arbres	2
Les arbres omniprésents en informatique	2
But	8
Définition itérative	8
Définition récursive	8
Arbres binaires	9
Intérêt des arbres binaires	9
Terminologies	10
Exemple	11
Définition récursive d'Arbre binaire	12
Opérations de base	15
Exemple de création d'arbre	19
Premier Version de création d'un arbre binaire (feuilles Vers Racine)	20
Deuxième Version de création d'un arbre binaire (Racine aux feuilles)	21
Parcours d'un arbre binaire	22
Parcours préfixe (pré-ordre)	23
Parcours infixe	25
Parcours postfixe	27
Parcours en largeur : Parcours niveau par niveau	29
Taille d'un arbre binaire	30
Définition	30
Algorithme	31
Nombre de feuille	33
Version 1	36
Version 2	37
Recherche d'un élément dans un arbre binaire	38
Problème	38
Raisonnement	39
Suppression d'un arbre binaire	44
Raisonnement	44
Attention traduction en C Profondeur / Hauteur	45
Définition : Profondeur	46
Définition : hauteur	46
Application : évaluation d'expression parenthésée	49

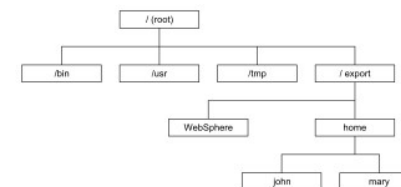
Arbres

Les arbres omniprésents en informatique

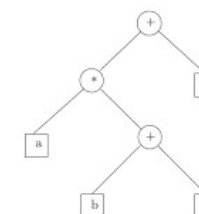
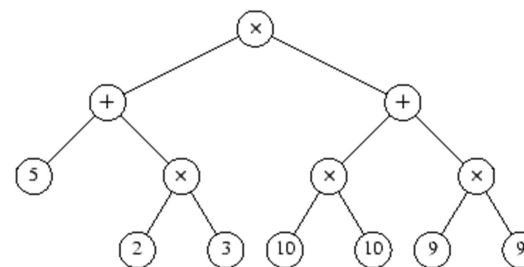
Dans les données



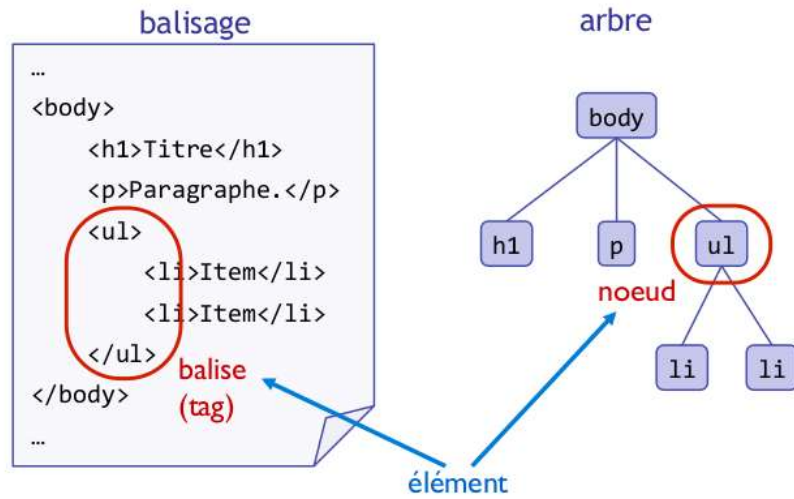
Système d'exploitation : organisation des répertoires et des fichiers



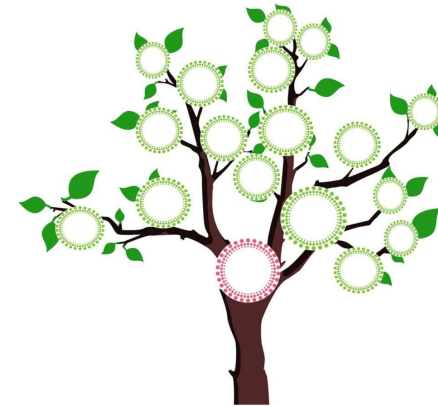
Expressions arithmétiques

FIG. 6 – Arbre de l'expression $a * (b + c) + d$.

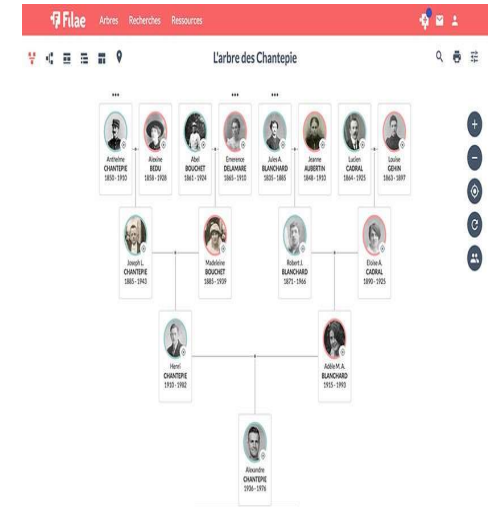
Document structuré (html, xml, tex...)



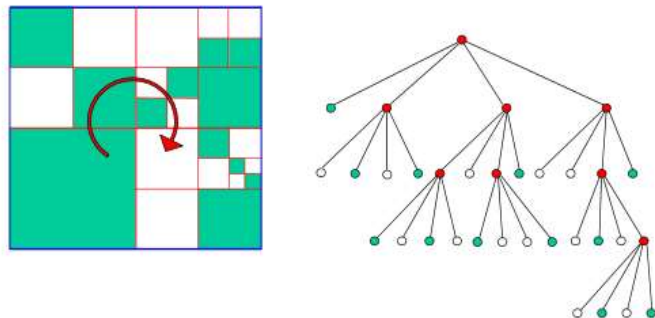
... et ailleurs (généalogie, organigrammes...)

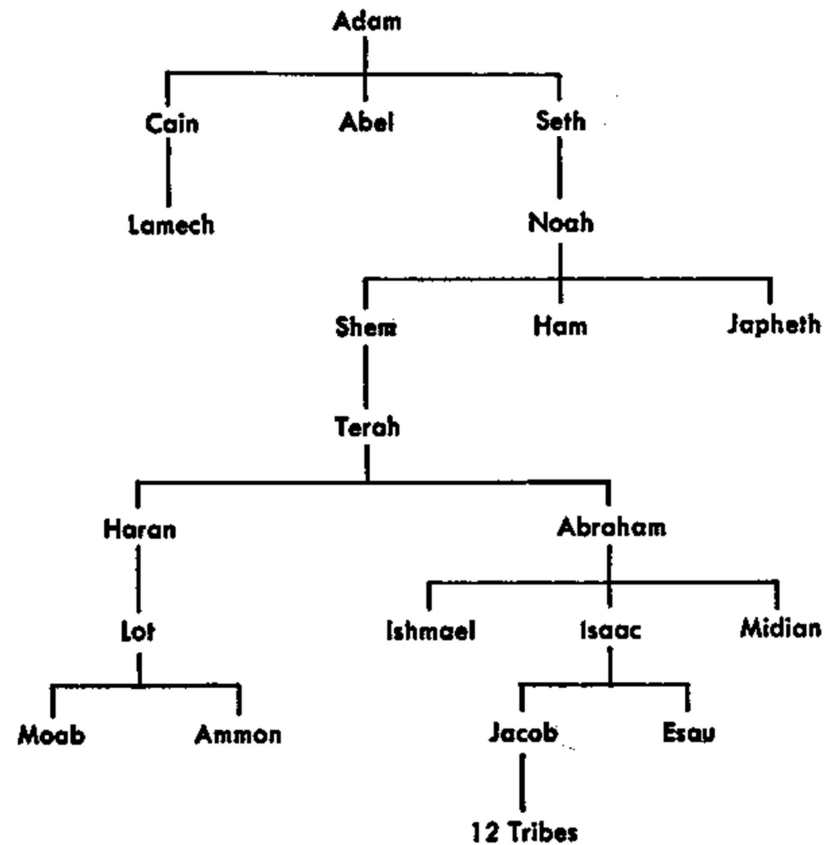
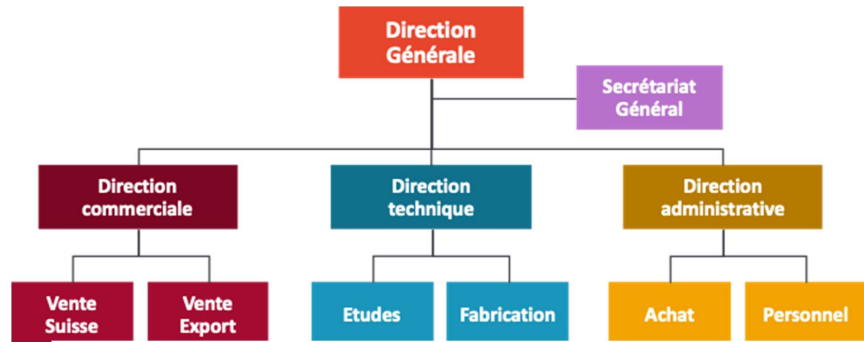


Mon Arbre Généalogique



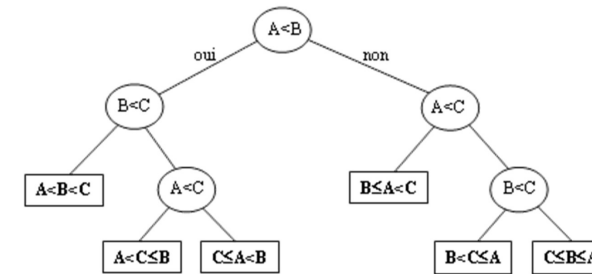
Représentation d'images (quadtree / octree)





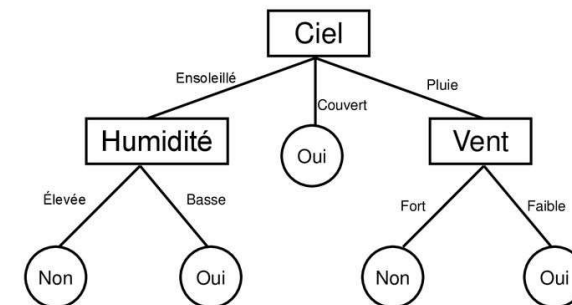
Dans les algorithmes

➤ Arbre de décision



Arbre de décision

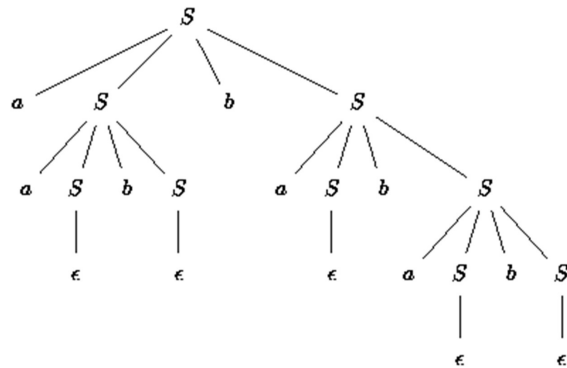
- Exemple : Est-ce que les conditions sont favorables pour jouer au tennis?



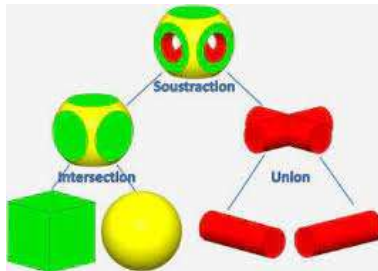
Classifier l'instance suivante:

<Ciel = Ensoleillé, Température = chaud, Humidité = élevé, Vent = fort>

Arbre d'analyse syntaxique



Arbre CSG : Modélisation en synthèse d'images



But

Représentation d'un ensemble fini d'éléments sous forme hiérarchique.

Définition itérative

Un **arbre** est un ensemble fini de **nœuds** de même type, qui sont liés entre eux par une relation de parenté, vérifiant les propriétés suivantes :

- Relation notée « X est le parent de Y » ou « Y est le fils de X »
- Il existe un unique nœud racine sans parent.
- Les autres nœuds de l'arbre possèdent un unique parent.

Définition récursive

Un arbre est défini (récursivement) par :

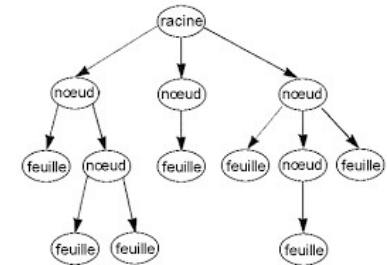
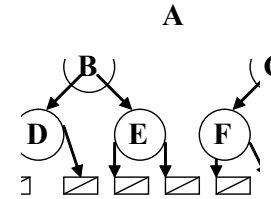
- Un premier nœud (appelé **racine** de l'arbre)
- 0, 1, 2 ou plusieurs **sous-arbres** disjoints (appelés **descendants** de la racine)

Arbres binaires

Intérêt des arbres binaires

Les arbres binaires sont intéressants parce qu'ils permettent d'optimiser le temps d'accès à une information : ils "mémoirisent" une dichotomie de rangement dans la structure de l'arbre.

Terminologies



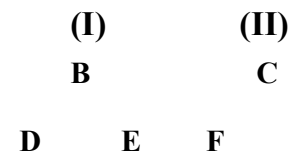
- Chaque nœud stocke une information (donnée) appelée **donnée** du nœud
- Chaque nœud est le **père** de ses fils
- Deux nœuds qui ont le même père sont **frères**

B est le nœud père de D et E → D et E sont frères.

A est le nœud racine : c'est un nœud sans père

L'arc qui relie 2 nœuds est appelé une **branche** : il représente la hiérarchie (Il existe toujours un arc entre deux nœuds cela définit la propriété de connexité).

Tous les nœuds sauf la racine n'ont qu'un seul père.



(I) et (II) Sont appelés des sous arbres gauches et sous arbre droit ; on parlera de **fils gauche** et de **fils droit**.

Exemple

Une expression arithmétique peut être représentée sous forme d'arbre :

Opérateurs = nœuds internes

Opérande = nœuds externes (feuilles)

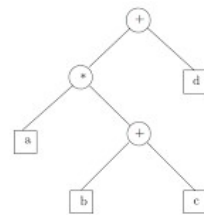
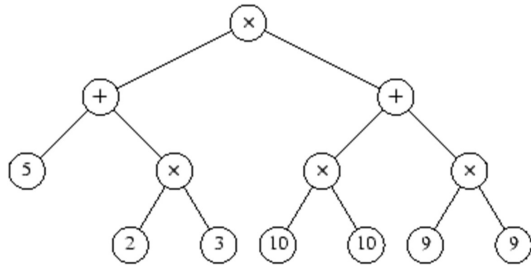
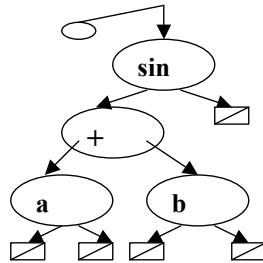


FIG. 6 – Arbre de l'expression $a * (b + c) + d$.

L'expression $\sin(a+b)$



Remarque

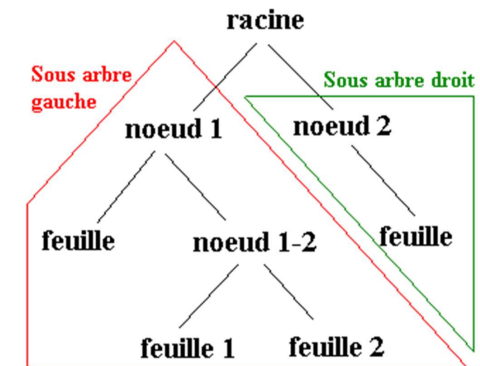
Nous avons remarqué que les schémas récursifs simplifiaient beaucoup l'écriture des algorithmes sur les listes chaînées.

Nous avons donné une définition récursive d'une liste chaînée. Il en est de même pour les arbres binaires.

Définition récursive d'Arbre binaire

Un arbre binaire est

- Soit vide
- Soit composé d'un élément appelé nœud auquel sont chaînés un sous arbre gauche et un sous arbre droit. Chaque sous arbre est aussi un arbre binaire.



Représentation chaînée d'un arbre binaire

```

Type      TElement = ....

Type      Arbre = Pointeur sur un Nœud

Type      Nœud = Enregistrement (Article ou structure)

                donnee : TElement

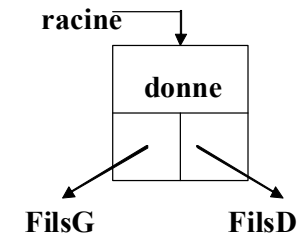
                filsG : arbre

                filsD : arbre

                Fin
    
```

En langage C

<pre> typedef --- TElement ; typedef struct noeud { TElement donnee; struct noeud *filsG; struct noeud *filsD; } * Arbre; </pre>	<pre> typedef --- TElement ; typedef struct noeud * Arbre; typedef struct noeud { TElement donnee; Arbre filsG; Arbre filsD; }; </pre>
Version 1	Version 2



Un arbre est donné par l'adresse de son premier nœud qu'on nomme racine (racine est de type arbre).

Primitives d'accès

Les actions qui permettent d'accéder à la donnée, au fils gauche et au fils droit d'un nœud non vide.

Soit a une variable de type arbre.

Si on suppose que l'arbre a est non vide alors :

Algorithmique

- a.donnee : accès au membre donnée de la structure arbre
- a.filsG : accès au membre fils gauche de la structure arbre
- a.filsD : accès au membre fils droit de la structure arbre

Langage C : a→ équivalent à (*a).

- a→donnee : accès au membre donnee de la structure arbre
- a→FilsG : accès au membre fils gauche de la structure arbre
- a→FilsD : accès au membre fils droit de la structure arbre

Opérations de base

Précondition : non estVide(racine)

Fonction filsGauche : Arbre → Arbre

Fonction filsGauche(racine) → Arbre

PF : racine de type arbre (E)

Début

Renvoyer racine.filsG

Fin

Fonction filsDroit : Arbre → Arbre

Fonction filsDroit(racine) → Arbre

PF : racine de type arbre (E)

Début

Renvoyer racine.filsD

Fin

Fonction donnee : Arbre → TElement

Fonction donnee(racine) → TElement

PF : racine de type arbre (E)

Début

Renvoyer racine.donnee

Fin

/* fonction création (d'initialisation) d'un arbre binaire */

Fonction initA : [] → Arbre

Fonction initA() → Arbre

Début

Renvoyer NULL

Fin

/* fonction des teste si un arbre binaire est vide */

Fonction estVide : Arbre → Booléenne

Fonction estVide(racine) → Booléenne

PF : racine de type arbre (E)

Début

Renvoyer racine=NULL

Fin

Fonction CreatFeuille : TElement → Arbre

Fonction CreatFeuille(elt) → Arbre

Debut

f ← allocMemoire(Arbre)

f.donnee ← elt

f.filsG ← initA()

f.filsD ← initA()

renvoyer f

FIN **V.I** f de type arbre

Fonction creatNoeud : TElementxArbrexArbre → Arbre

Fonction creatNoeud(elt, fg, fd) → Arbre

PF elt de type Tement (E)

Fg et fd sont de type Arbre (E)

Debut

a ← allocMemoire(Arbre)

a.donnee ← elt

a.filsG ← fg

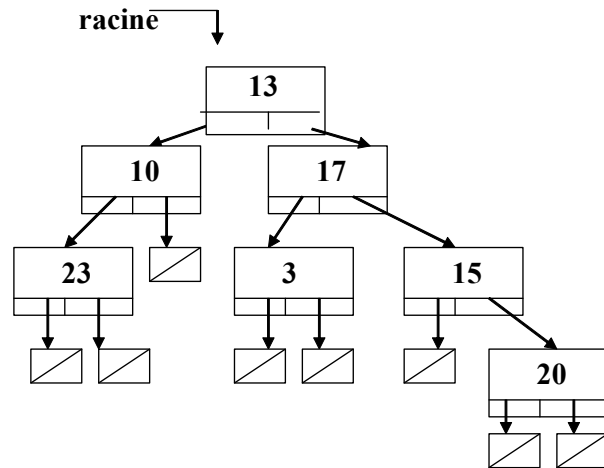
a.filsD ← fd

renvoyer a

Fin

V.I a de type arbre

Exemple de création d'arbre



Premier Version de création d'un arbre binaire (feuilles Vers Racine)

Fonction creatArbre : [] → Arbre

Fonction creatArbre() → Arbre

Début

```
F20 ← creatFeuille(20)
F23 ← creatFeuille(23)
F3 ← creatFeuille(3)
N15 ← creatNoeud(15, NULL, f20)
N10 ← creatNoeud(10, f23, NULL)
N17 ← creatNoeud(17, f3, f15)
Renvoyer creatNoeud(13, n10, n17)
```

Fin

VI : f20, f23, f3, n15, n17, n15 de type arbre

Deuxième Version de création d'un arbre

binaires (Racine aux feuilles)

Fonction `creatArbre()` → Arbre

Début

Voir TD

Fin

Parcours d'un arbre binaire

But :

Visiter l'ensemble des nœuds d'un arbre binaire afin d'effectuer une opération ou un traitement.

Par exemple, afficher la donnée associée à chaque nœud de l'arbre.

Il existe deux sortes de parcours

1. Parcours en profondeur

2. Parcours en largeur

On étudiera dans un premier temps le parcours en profondeur puis dans un second temps le parcours en largeur (voir TD).

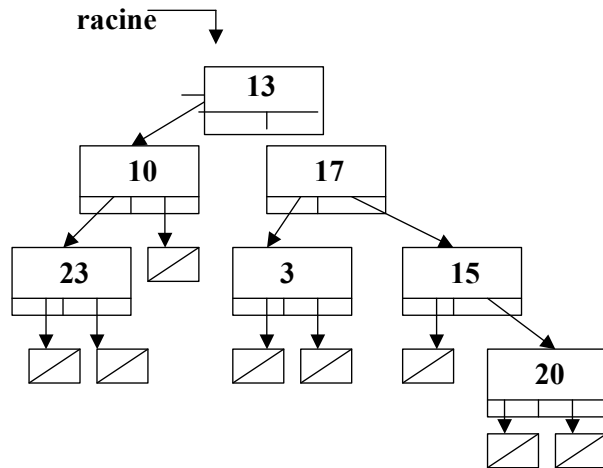
On distingue trois types de parcours en profondeur :

- Parcours préfixe
- Parcours infixé
- Parcours postfixé

Parcours préfixe (pré-ordre)

Ce parcours consiste à effectuer dans l'ordre :

- Traitement de la racine
- Parcours du fils gauche
- Parcours du fils droit



Exemple :

Parcours préfixe de l'arbre ci-dessous est : [13, 10, 23, 17, 3, 15, 20]

Procédure parcours Prefixe(racine)

P.F racine est de type arbre (E)

Début

Si non estVideArbre(racine) alors

traiter(donnee(racine))

parcoursPrefixe(filsGauche(racine))

parcoursPrefixe(filsDroit(racine))

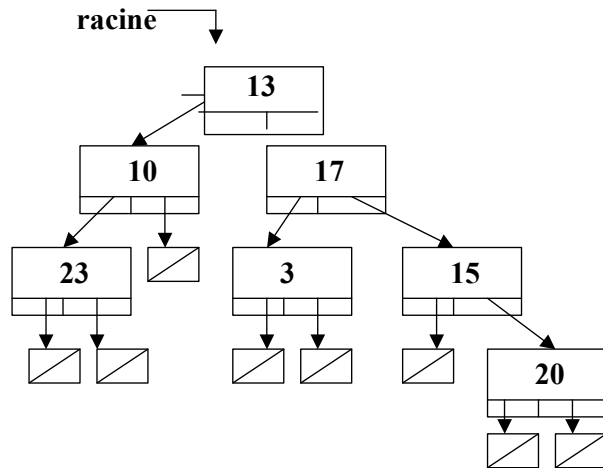
finsi

Fin

Parcours infixe

Ce parcours consiste à effectuer dans l'ordre :

- Parcours du fils gauche
- Traitement de la racine
- Parcours du fils droit



Exemple :

Parcours infixe de l'arbre ci-dessous est : [23, 10, 13, 3, 17, 15, 20]

Procédure **parcoursInfixe(racine)**

P.F racine est de type arbre (E)

Début

```

Si non estVideArbre(racine) alors

    parcoursInfixe(filsGauche(racine))

    traiter(donnee(racine))

    parcoursInfixe(filsDroit(racine))

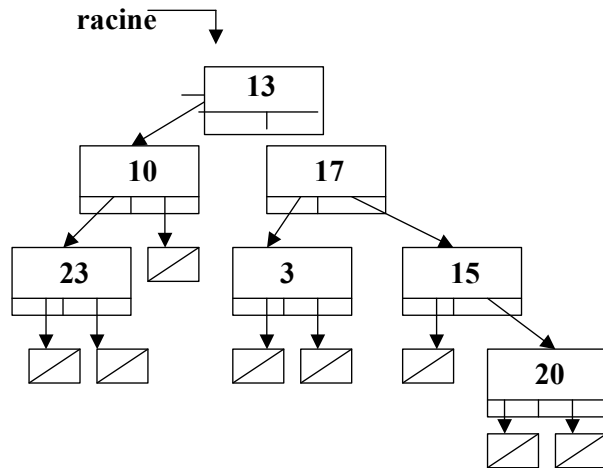
finsi
  
```

Fin

Parcours postfixe

Ce parcours consiste à effectuer dans l'ordre :

- Parcours du fils gauche
- Parcours du fils droit
- Traitement de la racine



Exemple :

Parcours postfixe de l'arbre ci-dessous est : [23, 10, 3, 20, 15, 17, 13]

Procédure **parcoursPostfixe(racine)**

P.F racine est de type arbre (E)

Début

Si non estVideArbre(racine) alors

parcoursPostfixe(filsGauche(racine))

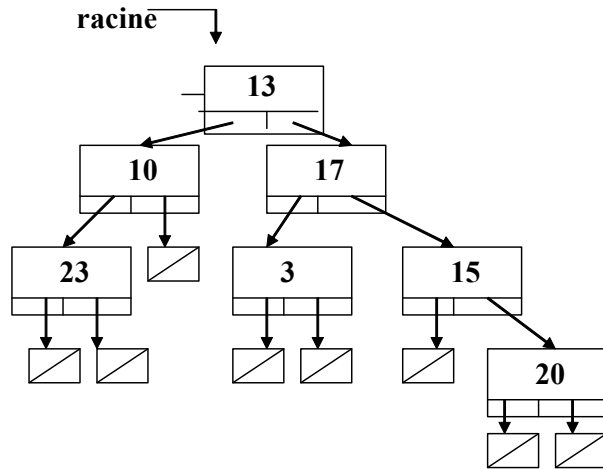
parcoursPostfixe(filsDroit(racine))

traiter(donnee(racine))

finsi

Fin

Parcours en largeur : Parcours niveau par niveau



Le parcours en largeur de l'arbre est : 13-10-17-23-3-15-20

Algorithme du parcours en largeur Voir TD

Taille d'un arbre binaire

Définition

On appelle taille d'un arbre binaire le nombre de nœuds de cet arbre

Algorithme

Fonction `tailleArbre` : `Arbre` $\rightarrow$ `Entier`

Raisonnement :

Si a est vide :

Le nombre de nœuds est égal à zéro

L'algorithme est terminé

Si a est un arbre non vide :

Cet arbre est donc composé d'un élément nœud chaîné d'un sous arbre gauche et d'un sous arbre droit

La taille de l'arbre est donc égale à la taille du nœud (donc 1) plus la taille du sous arbre gauche plus la taille du sous arbre droit

Fonction `tailleArbre(racine)` $\rightarrow$ `Entier`

F.P `racine` : `Arbre (E)`

Début

 Si `estVide(racine)` alors

 Renvoyer 0

 Sinon

 Renvoyer 1 +

`tailleArbre(filsGauche(racine))`

 + `tailleArbre(filsDroit(racine))`

 Finsi

Fin

Nombre de feuille

Calculer le nombre de feuilles présent dans un arbre binaire

Fonction nbFeuilles :Arbre $\rightarrow$ Entier

Rappel : une feuille est un nœud sans fils gauche ni fils droit.

Fonction estFeuille : Arbre $\rightarrow$ Booléenne

Fonction estFeuille(racine) $\rightarrow$ Booléenne

Début

Renvoyer estVide(filsGauche(racine)) et estVide(filsDroit (racine))

Fin

Raisonnement

Fonction nbFeuilles : Arbre $\rightarrow$ Entier

Si l'arbre est vide

Le nombre de feuilles est égal à zéro

L'algorithme est terminé (arrêt de la récursivité)

Si l'arbre n'est pas vide : on distingue deux cas :

Si l'arbre est réduit à une feuille $\rightarrow$ le nombre de feuilles est 1

Si l'arbre n'est pas réduit à une feuille $\rightarrow$ le nombre de feuilles de l'arbre est égale au nombre de feuilles du sous arbre gauche plus le nombre de feuille du sous arbre droit.

Version 1

Fonction nbFeuilles(a) $\rightarrow$ Entier

P.F a type arbre (E)

Début

Si estVide(a) alors

Renvoyer 0

Sinon

Si estFeuille(a) alors

Renvoyer 1

Sinon

Renvoyer nbFeuilles(filsGauche(a))

+ nbFeuilles(filsDroit(a))

Finsi

Finsi

Fin

Version 2

```
Fonction nbFeuilles(a) → Entier  
  
P.F a   type arbre (E)  
  
Début  
  
    Si estVide(a) alors  
  
        renvoyer 0  
  
    FinSi  
  
    Si estFeuille(a) alors  
  
        renvoyer 1  
  
    FinSi  
  
    renvoyer nbFeuilles(filsGauche(a))  
           + nbFeuilles(filsDroit(a))  
  
Fin
```

Recherche d'un élément dans un arbre binaire

Problème

On souhaite tester l'existence de la première occurrence d'un élément donné dans un arbre binaire quelconque.

```
Fonction rechEltArbre : TElementxArbre-> Booléenne
```

Raisonnement

Si l'arbre est vide

L'élément recherché n'existe pas dans l'arbre

Fonction retourne faux

Si l'arbre n'est pas vide : on distingue deux cas :

- Soit la donnée du premier nœud de l'arbre (racine) est égale à l'élément recherché auquel cas on arrête le traitement et la fonction retourne vrai
- Sinon, on recherche l'élément dans le sous arbre gauche et s'il n'existe pas on relance la recherche dans le sous arbre droit.

Version 1

Fonction `rechEltArbre(elt, a) -> Booléenne`

P.F `elt` : type `Telement (E)`. `a` : type `arbre (E)`

Début

`Si estVide(a)`

`Renvoyer faux`

`Sinon`

`si donnee(a)=elt alors`

`Renvoyer vrai`

`Sinon`

`Si` **`rechEltArbre(elt, filsGauche(a)) alors`**

`Renvoyer vrai`

`Sinon`

`renvoyer` **`rechEltArbre(elt, filsDroit(a))`**

`finsi`

`Finsi`

`finsi`

`Fin`

Version 2

Fonction rechEltArbre(elt, a) -> Booléenne

P.F elt : type Telement (E). a : type arbre (E)

Début

Si estVide(a)

renvoyer faux

Sinon

si donnee(a)=elt alors

renvoyer vrai

Sinon

renvoyer **rechEltArbre(elt, filsGauche(a)) OU**

rechEltArbre(elt, filsDroit(a))

Finsi

finsi

Fin

Version 3

Fonction rechEltArbre(elt, a) -> Booléenne

P.F elt : type Telement (E). a : type arbre (E)

Début

Si estVide(a)

renvoyer faux

Sinon

renvoyer donnee(a)=elt ou

rechEltArbre(elt, filsGauche(a)) **OU**

rechEltArbre(elt, filsDroit(a))

finsi

Fin

```
Fonction rechEltArbre(elt, a) -> Booléenne  
  
P.F  elt : type Telement (E).    a : type arbre (E)  
  
Début  
  
Si estVide(a)  
  
    renvoyer faux  
  
FinSi  
  
renvoyer donnee(a)=elt ou  
  
    rechEltArbre(elt, filsGauche(a)) OU  
  
    rechEltArbre(elt, filsDroit(a))  
  
Fin
```

Suppression d'un arbre binaire

Raisonnement

Si l'arbre est vide :

On ne fait rien = arrêt de la récursivité

Si l'arbre n'est pas vide :

On supprime d'abord le fils gauche et le fils droit puis on supprime en fin la racine.

Procédure `suppArbre(racine)`

PF : racine de type arbre (E/S)

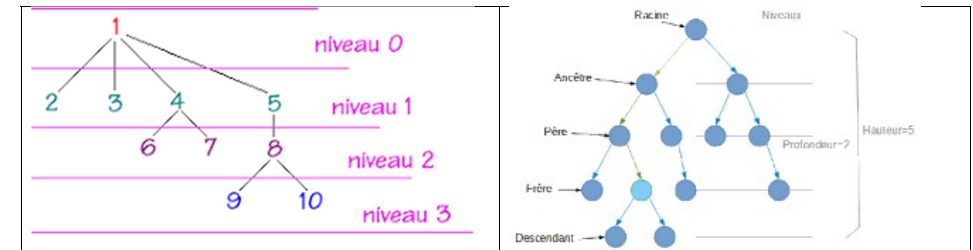
Début

```
Si non estVide(racine) alors  
    SuppArbre(filsGauche(racine))  
    SuppArbre(filsDroit(racine))  
    LibMémoire(racine)  
Finsi
```

Fin

Attention traduction en C

Profondeur / Hauteur



Définition : Profondeur

La profondeur d'un nœud est la longueur du chemin allant de la racine à ce nœud.

La profondeur de la racine est nulle et la profondeur d'un nœud autre que la racine vaut un plus la profondeur de son père.

La profondeur d'un nœud peut aussi être appelée niveau ou hauteur du nœud.

Définition : hauteur

La hauteur d'un arbre est la profondeur maximum de ses nœuds.

Version 1

Fonction hauteurArbre(racine) → Entier

Début

si estVideArbre(racine) alors

renvoyer -1 // par convention

sinon

si estFeuille(racine) alors

renvoyer 0

sinon

hg ← hauteurArbre(filsGauche(racine))

hd ← hauteurArbre(filsDroit(racine))

renvoyer 1 + max(hg, hd)

Finsi

Finsi

Fin

Version 2

Fonction hauteurArbre(racine) → Entier

Début

si estVideArbre(racine) alors

renvoyer -1 // par convention

FinSi

si estFeuille(racine) alors

renvoyer 0

FinSi

hg ← hauteurArbre(filsGauche(racine))

hd ← hauteurArbre(filsDroit(racine))

renvoyer 1 + max(hg, hd)

Fin

Application : évaluation d'expression parenthésée

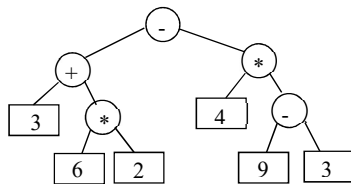
Soit une expression arithmétique complètement parenthésée, constituée d'opérandes, d'opérateurs binaires ('+', '-', '*', '/'), de parenthèses ('(', ')') et d'espaces.

Pour ce faire, on transforme l'expression sous forme d'arbre puis on évalue l'arbre ainsi obtenu en utilisant un parcours post-fixé.

A) Pour simplifier, on suppose que les opérandes sont des entiers.

Exemple : $exp=((3+(6*2))-(4*(9-3)))$ est une chaîne de caractères

Sa représentation en arbre binaire donne l'arbre binaire ci-dessous :



- 1) Transformer l'expression complètement parenthésée donnée en une expression post-fixée équivalente représentée par un arbre.
- 2) Evaluer l'expression post-fixée.